# Human review packet: PRPKG24AccuracyDiversity

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

**Paper:** PRPKG24AccuracyDiversity
**Paper id:** `PRPKG24AccuracyDiversity`
**Source version:** not recorded
**Source record:** `audit/paper_statement_map.json`
**Rows:** 25
**Generated:** 2026-08-18
**Source URL:** official source

**Review status:** 0/25 source-claim screens; 0/47 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.

## How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

## Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper PRPKG24AccuracyDiversity --serve
```

Open `http://127.0.0.1:8765/` in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

## Contents

- EconCSLib.Probability.TopKExpectationOracle.common
  - EconCSLib.Probability.TopKExpectationOracle.expectedTopSum
  - EconCSLib.Probability.orderStatisticTopKSumFromMean
  - EconCSLib.Probability.TopKExpectationOracle.orderStatisticTopKExpectationOracle
  - EconCSLib.Probability.ascendingOrderStatistic
  - EconCSLib.Probability.binaryRatingScore
  - EconCSLib.Probability.sampleOrderStatisticValue
  - EconCSLib.Probability.expectedSampleOrderStatisticMean
  - EconCSLib.Probability.expectedOrderStatisticMeanSeq
  - EconCSLib.Probability.topKRankEmbedding
  - EconCSLib.Probability.upperOrderStatistic
  - EconCSLib.Probability.sampleTopKSum
  - EconCSLib.Probability.expectedSampleTopKSum
  - EconCSLib.Probability.realBernoulliPMF
  - EconCSLib.Probability.reflectedCDFMass
  - EconCSLib.finiteMin
  - EconCSLib.pmfExp
  - EconCSLib.pmfPi
  - EconCSLib.pmfProb
  - EconCSLib.pmfProduct
- Source claims (25)
  - theorem1_i_formulaSpec
  - theorem1_ii_formulaSpec
  - theorem1_iii_formulaSpec
  - theorem1_iv_formulaSpec
  - theorem1_v_selected_maximizer_conclusionSpec
  - corollary1Spec
  - proposition2_corrected_finite_and_sequence_sourceSpec
  - corollary3_iid_bernoulli_conclusionSpec
  - theorem2_i_corrected_pmf_source_model_endpointSpec
  - theorem2_ii_corrected_pmf_source_model_endpointSpec
  - theorem2_iii_corrected_pmf_source_model_endpointSpec
  - theorem2_iv_corrected_pmf_source_model_endpointSpec
  - theorem3_log_share_conclusionSpec
  - theorem3_all_consumed_argmax_conclusionSpec
  - lemma1_bounded_topk_loss_asymptotic_of_pdf_sourceSpec
  - proposition4Spec
  - lemmaD1_i_corrected_uniform_optimizer_shares_sourceSpec
  - lemmaD1_ii_corrected_powerTail_optimizer_shares_sourceSpec
  - lemmaD1_iii_corrected_probability_optimizer_shares_sourceSpec
  - lemmaD1_iv_corrected_powerTail_optimizer_shares_sourceSpec
  - lemmaD2_bounded_fixed_rank_integral_asymptotic_of_pdf_sourceSpec
  - lemmaD3_corrected_exponential_fixed_rank_sourceSpec
  - lemmaD4_corrected_pareto_fixed_rank_sourceSpec
  - lemmaD5Spec
  - proposition5_iid_topk_identitySpec

# Material library prerequisites

## `EconCSLib.Allocation`

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean declaration type (metadata)

```
Type u_1 → Type u_1
```

### Exact Lean library declaration

```
structure Allocation (κ : Type*) where
  count : κ → ℕ
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.count`

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{κ : Type u_1} → (self : EconCSLib.Allocation κ) → (a : κ) → self.1 a
```

### Exact Lean library declaration

```
library declaration is not registered or discoverable
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.total`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_1} → [inst : Fintype κ] → (a : EconCSLib.Allocation κ) → ∑ k, a.count k
```

## Exact Lean library declaration

```
def total (a : Allocation κ) : ℕ :=
  ∑ k, a.count k
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.HasTotal`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
∀ {κ : Type u_1} [inst : Fintype κ] (a : EconCSLib.Allocation κ) (N : ℕ), a.total = N
```

## Exact Lean library declaration

```
def HasTotal (a : Allocation κ) (N : ℕ) : Prop :=
  a.total = N
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

| |
|---|
| |

**EconCSLib.Allocation.OptimalSequence**

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
{κ : Type u_1} → [Fintype κ] → (ℕ → κ → ℝ) → (ℕ → κ → ℕ → ℝ) → Type u_1
```

## Exact Lean library declaration

```
structure OptimalSequence
    (objectiveWeightSeq : ℕ → κ → ℝ)
    (valueOfCountSeq : ℕ → κ → ℕ → ℝ) where
  allocation : ℕ → Allocation κ
  optimal : ∀ N,
    IsOptimalAtTotal (objectiveWeightSeq N) (valueOfCountSeq N) N (allocation N)
```

## Recorded source-to-library screening  Verdict: `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.OptimalSequence.allocation`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_1} →
  [inst : Fintype κ] →
    {objectiveWeightSeq : ℕ → κ → ℝ} →
      {valueOfCountSeq : ℕ → κ → ℕ → ℝ} →
        (self : EconCSLib.Allocation.OptimalSequence objectiveWeightSeq valueOfCountSeq) → (a : ℕ) → self.1 a
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.Sequence`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
(κ : Type u_2) → [Fintype κ] → Type u_2
```

## Exact Lean library declaration

```
structure Sequence (κ : Type*) [Fintype κ] where
  allocation : ℕ → Allocation κ
  feasible : ∀ N, HasTotal (allocation N) N
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.Sequence.mk`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
{κ : Type u_2} →
  [inst : Fintype κ] →
    (allocation : ℕ → EconCSLib.Allocation κ) → (∀ (N : ℕ), (allocation N).HasTotal N) → EconCSLib.Allocation.Sequence κ
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

### `EconCSLib.Allocation.OptimalSequence.toSequence`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_1} →
  [inst : Fintype κ] →
    {objectiveWeightSeq : ℕ → κ → ℝ} →
      {valueOfCountSeq : ℕ → κ → ℕ → ℝ} →
        (seq : EconCSLib.Allocation.OptimalSequence objectiveWeightSeq valueOfCountSeq) →
          { allocation := seq.allocation, feasible := ⋯ }
```

## Exact Lean library declaration

```
def toSequence
    {objectiveWeightSeq : ℕ → κ → ℝ}
    {valueOfCountSeq : ℕ → κ → ℕ → ℝ}
    (seq : OptimalSequence objectiveWeightSeq valueOfCountSeq) :
    Sequence κ where
  allocation := seq.allocation
  feasible := fun N => (seq.optimal N).1
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.Sequence.allocation`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_2} → [inst : Fintype κ] → (self : EconCSLib.Allocation.Sequence κ) → (a : ℕ) → self.1 a
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## EconCSLib.Allocation.share

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{κ : Type u_1} →
  [inst : Fintype κ] → (a : EconCSLib.Allocation κ) → (k : κ) → if h : a.total = 0 then 0 else ↑(a.count k) / ↑a.total
```

### Exact Lean library declaration

```
noncomputable def share (a : Allocation κ) (k : κ) : ℝ :=
  if h : a.total = 0 then 0 else (a.count k : ℝ) / (a.total : ℝ)
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### EconCSLib.Allocation.Sequence.share

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_1} →
  [inst : Fintype κ] → (seq : EconCSLib.Allocation.Sequence κ) → (N : ℕ) → (k : κ) → (seq.allocation N).share k
```

## Exact Lean library declaration

```
noncomputable def share (seq : Sequence κ) (N : ℕ) (k : κ) : ℝ :=
  Allocation.share (seq.allocation N) k
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.Sequence.ConvergesToProfile`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
∀ {κ : Type u_1} [inst : Fintype κ] (seq : EconCSLib.Allocation.Sequence κ) (target : κ → ℝ) (k : κ),
  Filter.Tendsto (fun N => seq.share N k) Filter.atTop (nhds (target k))
```

## Exact Lean library declaration

```
def ConvergesToProfile (seq : Sequence κ) (target : κ → ℝ) : Prop :=
  ∀ k, Tendsto (fun N => seq.share N k) atTop (nhds (target k))
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.mk`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
{κ : Type u_1} → (κ → ℕ) → EconCSLib.Allocation κ
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### EconCSLib.Allocation.allOnTypeAllocation

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_1} → [inst : DecidableEq κ] → (N : ℕ) → (best : κ) → { count := fun k => if k = best then N else 0 }
```

## Exact Lean library declaration

```
def allOnTypeAllocation (N : ℕ) (best : κ) : Allocation κ where
  count := fun k => if k = best then N else 0
```

## Recorded source-to-library screening  **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Allocation.objective`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{κ : Type u_1} →
  [inst : Fintype κ] →
    (a : EconCSLib.Allocation κ) →
      (weight : κ → ℝ) → (valueOfCount : κ → ℕ → ℝ) → ∑ k, weight k * valueOfCount k (a.count k)
```

## Exact Lean library declaration

```
noncomputable def objective
    (a : Allocation κ) (weight : κ → ℝ) (valueOfCount : κ → ℕ → ℝ) : ℝ :=
  ∑ k, weight k * valueOfCount k (a.count k)
```

## Recorded source-to-library screening  **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Math.AsymptoticEquivalent`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
∀ (x y : ℕ → ℝ), Filter.Tendsto (fun n => x n / y n) Filter.atTop (nhds 1)
```

## Exact Lean library declaration

```
def AsymptoticEquivalent (x y : ℕ → ℝ) : Prop :=
  Tendsto (fun n => x n / y n) atTop (nhds 1)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Math.TendsToZero`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
∀ (ε : ℕ → ℝ), Filter.Tendsto ε Filter.atTop (nhds 0)
```

## Exact Lean library declaration

```
def TendsToZero (ε : ℕ → ℝ) : Prop :=
  Tendsto ε atTop (nhds 0)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.Exponential.Model`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
Type
```

## Exact Lean library declaration

```
structure Model where
  rate : ℝ
  rate_pos : 0 < rate
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.Exponential.Model.rate`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(self : EconCSLib.Probability.Exponential.Model) → self.1
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.Exponential.Model.measure`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(M : EconCSLib.Probability.Exponential.Model) → ProbabilityTheory.expMeasure M.rate
```

## Exact Lean library declaration

```
noncomputable def measure (M : Model) : MeasureTheory.Measure ℝ :=
  ProbabilityTheory.expMeasure M.rate
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.Exponential.Model.iidProductMeasure`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(M : EconCSLib.Probability.Exponential.Model) → (q : ℕ) → MeasureTheory.Measure.pi fun x => M.measure
```

## Exact Lean library declaration

```
noncomputable def Model.iidProductMeasure
    (M : Model) (q : ℕ) : MeasureTheory.Measure (Fin q → ℝ) :=
  MeasureTheory.Measure.pi (fun _ : Fin q => M.measure)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

| |
|---|
| |

### `EconCSLib.Probability.Exponential.Model.mk`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
(rate : ℝ) → 0 < rate → EconCSLib.Probability.Exponential.Model
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### EconCSLib.Probability.Exponential.harmonicReal

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(q : ℕ) → ∑ j ∈ Finset.range q, 1 / ↑(j + 1)
```

## Exact Lean library declaration

```
noncomputable def harmonicReal (q : ℕ) : ℝ :=
  ∑ j ∈ Finset.range q, (1 : ℝ) / ((j + 1 : ℕ) : ℝ)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.Pareto.iidProductMeasure`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(t r : ℝ) → (q : ℕ) → MeasureTheory.Measure.pi fun x => ProbabilityTheory.paretoMeasure t r
```

## Exact Lean library declaration

```
noncomputable def iidProductMeasure (t r : ℝ) (q : ℕ) :
    MeasureTheory.Measure (Fin q → ℝ) :=
  MeasureTheory.Measure.pi
    (fun _ : Fin q => ProbabilityTheory.paretoMeasure t r)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### EconCSLib.Probability.TopKExpectationOracle

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
Type u_1 → Type u_1
```

## Exact Lean library declaration

```
structure TopKExpectationOracle (τ : Type*) where
  expectedTopSum : ℕ → τ → ℕ → ℝ
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.TopKExpectationOracle.mk`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean declaration type (metadata)

```
{τ : Type u_1} → (ℕ → τ → ℕ → ℝ) → EconCSLib.Probability.TopKExpectationOracle τ
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.TopKExpectationOracle.common`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(τ : Type u_2) → (h : ℕ → ℝ) → { expectedTopSum := fun _k _t q => h q }
```

## Exact Lean library declaration

```
def common (τ : Type*) (h : ℕ → ℝ) : TopKExpectationOracle τ where
  expectedTopSum := fun _k _t q => h q
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.TopKExpectationOracle.expectedTopSum`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{τ : Type u_1} →
  (self : EconCSLib.Probability.TopKExpectationOracle τ) → (a : ℕ) → (a_1 : τ) → (a_2 : ℕ) → self.1 a a_1 a_2
```

## Exact Lean library declaration

```
library declaration is not registered or discoverable
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.orderStatisticTopKSumFromMean`

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
(μ : ℕ → ℕ → ℝ) → (k a : ℕ) → ∑ i ∈ Finset.range (min k a), μ (a - i) a
```

### Exact Lean library declaration

```
def orderStatisticTopKSumFromMean
    (μ : ℕ → ℕ → ℝ) (k a : ℕ) : ℝ :=
  ∑ i ∈ Finset.range (min k a), μ (a - i) a
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

**`EconCSLib.Probability.TopKExpectationOracle.orderStatisticTopKExpectationOracle`**

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
(τ : Type u_2) →
  (μ : ℕ → ℕ → ℝ) → { expectedTopSum := fun k _t q => EconCSLib.Probability.orderStatisticTopKSumFromMean μ k q }
```

### Exact Lean library declaration

```
def orderStatisticTopKExpectationOracle
    (τ : Type*) (μ : ℕ → ℕ → ℝ) : TopKExpectationOracle τ where
  expectedTopSum := fun k _t q => orderStatisticTopKSumFromMean μ k q
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

**`EconCSLib.Probability.ascendingOrderStatistic`**

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{n : ℕ} → (sample : Fin n → ℝ) → (rank : Fin n) → sample ((Tuple.sort sample) rank)
```

### Exact Lean library declaration

```
def ascendingOrderStatistic {n : ℕ} (sample : Fin n → ℝ) (rank : Fin n) : ℝ :=
  sample (Tuple.sort sample rank)
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

**`EconCSLib.Probability.binaryRatingScore`**

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
(b : Bool) → if b = true then 1 else 0
```

### Exact Lean library declaration

```
def binaryRatingScore (b : Bool) : ℝ :=
  if b then 1 else 0
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

`EconCSLib.Probability.sampleOrderStatisticValue`

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{a : ℕ} →
  (sample : Fin a → ℝ) →
    (rank : ℕ) →
      if hrank : 0 < rank ∧ rank ≤ a then EconCSLib.Probability.ascendingOrderStatistic sample ⟨rank - 1, ⋯⟩ else 0
```

### Exact Lean library declaration

```
def sampleOrderStatisticValue {a : ℕ}
    (sample : Fin a → ℝ) (rank : ℕ) : ℝ :=
  if hrank : 0 < rank ∧ rank ≤ a then
    ascendingOrderStatistic sample ⟨rank - 1, by omega⟩
  else 0
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.expectedSampleOrderStatisticMean`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{a : ℕ} →
  (μ : MeasureTheory.Measure (Fin a → ℝ)) →
    (rank sampleSize : ℕ) →
      if sampleSize = a then ∫ (sample : Fin a → ℝ), EconCSLib.Probability.sampleOrderStatisticValue sample rank ∂μ
      else 0
```

## Exact Lean library declaration

```
def expectedSampleOrderStatisticMean {a : ℕ}
    (μ : MeasureTheory.Measure (Fin a → ℝ)) (rank sampleSize : ℕ) : ℝ :=
  if sampleSize = a then
    ∫ sample, sampleOrderStatisticValue sample rank ∂μ
  else 0
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.expectedOrderStatisticMeanSeq`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(sampleMeasure : (a : ℕ) → MeasureTheory.Measure (Fin a → ℝ)) →
  (rank sampleSize : ℕ) →
    EconCSLib.Probability.expectedSampleOrderStatisticMean (sampleMeasure sampleSize) rank sampleSize
```

## Exact Lean library declaration

```
def expectedOrderStatisticMeanSeq
    (sampleMeasure : (a : ℕ) → MeasureTheory.Measure (Fin a → ℝ))
    (rank sampleSize : ℕ) : ℝ :=
  expectedSampleOrderStatisticMean (sampleMeasure sampleSize) rank sampleSize
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### EconCSLib.Probability.topKRankEmbedding

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(k n : ℕ) → (i : Fin (min k n)) → ⟨↑i, ⋯⟩
```

## Exact Lean library declaration

```
def topKRankEmbedding (k n : ℕ) (i : Fin (min k n)) : Fin n :=
  ⟨i.val, lt_of_lt_of_le i.isLt (min_le_right k n)⟩
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.upperOrderStatistic`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{n : ℕ} →
  (sample : Fin n → ℝ) → (rankFromTop : Fin n) → EconCSLib.Probability.ascendingOrderStatistic sample rankFromTop.rev
```

## Exact Lean library declaration

```
def upperOrderStatistic {n : ℕ} (sample : Fin n → ℝ)
    (rankFromTop : Fin n) : ℝ :=
  ascendingOrderStatistic sample rankFromTop.rev
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.sampleTopKSum`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{n : ℕ} →
  (sample : Fin n → ℝ) →
    (k : ℕ) → ∑ i, EconCSLib.Probability.upperOrderStatistic sample (EconCSLib.Probability.topKRankEmbedding k n i)
```

## Exact Lean library declaration

```
def sampleTopKSum {n : ℕ} (sample : Fin n → ℝ) (k : ℕ) : ℝ :=
  ∑ i : Fin (min k n),
    upperOrderStatistic sample (topKRankEmbedding k n i)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

`EconCSLib.Probability.expectedSampleTopKSum`

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{n : ℕ} →
  (μ : MeasureTheory.Measure (Fin n → ℝ)) →
    (k : ℕ) → ∫ (sample : Fin n → ℝ), EconCSLib.Probability.sampleTopKSum sample k ∂μ
```

### Exact Lean library declaration

```
def expectedSampleTopKSum {n : ℕ}
    (μ : MeasureTheory.Measure (Fin n → ℝ)) (k : ℕ) : ℝ :=
  ∫ sample, sampleTopKSum sample k ∂μ
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.realBernoulliPMF`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(p : ℝ) → (hp0 : 0 ≤ p) → (hp1 : p ≤ 1) → PMF.bernoulli ⟨p, hp0⟩ ⋯
```

## Exact Lean library declaration

```
def realBernoulliPMF (p : ℝ) (hp0 : 0 ≤ p) (hp1 : p ≤ 1) : PMF Bool :=
  PMF.bernoulli ⟨p, hp0⟩ (by
    change p ≤ (1 : ℝ)
    exact hp1)
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.Probability.reflectedCDFMass`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
(μ : MeasureTheory.Measure ℝ) → (M x : ℝ) → μ.real {y | M ≤ x + y}
```

## Exact Lean library declaration

```
def reflectedCDFMass (μ : Measure ℝ) (M x : ℝ) : ℝ :=
  μ.real {y : ℝ | M ≤ x + y}
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

### `EconCSLib.finiteMin`

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{α : Type u_1} → [inst : Fintype α] → [inst_1 : Nonempty α] → (f : α → ℝ) → Finset.univ.inf' ⋯ f
```

## Exact Lean library declaration

```
def finiteMin {α : Type*} [Fintype α] [Nonempty α]
    (f : α → ℝ) : ℝ :=
  (Finset.univ : Finset α).inf' Finset.univ_nonempty f
```

## Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

| |
|---|
| |

### EconCSLib.pmfExp

**Source connection:** no explicit byte-pinned paper source connection is registered.

## Lean-expanded library semantic target

```
{α : Type u_1} → [inst : Fintype α] → [DecidableEq α] → (μ : PMF α) → (f : α → ℝ) → ∑ a, (μ a).toReal * f a
```

## Exact Lean library declaration

```
noncomputable def pmfExp {α : Type*} [Fintype α] [DecidableEq α]
    (μ : PMF α) (f : α → ℝ) : ℝ :=
  ∑ a, (μ a).toReal * f a
```

## Recorded source-to-library screening   Verdict: `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

**`EconCSLib.pmfPi`**

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{ι : Type u_1} →
  [inst : Fintype ι] →
    [inst_1 : DecidableEq ι] →
      {α : ι → Type u_2} →
        [inst_2 : (i : ι) → Fintype (α i)] → (μ : (i : ι) → PMF (α i)) → PMF.ofFintype (fun f => ∏ i, (μ i) (f i)) ⋯
```

### Exact Lean library declaration

```
noncomputable def pmfPi {ι : Type*} [Fintype ι] [DecidableEq ι]
    {α : ι → Type*} [∀ i : ι, Fintype (α i)]
    (μ : ∀ i : ι, PMF (α i)) : PMF ((i : ι) → α i) :=
  PMF.ofFintype (fun f : (i : ι) → α i => ∏ i : ι, μ i (f i)) (by
    classical
    have hcoord : ∀ i : ι, ∑ a : α i, μ i a = 1 := by
      intro i
      rw [← PMF.tsum_coe (μ i), tsum_fintype]
    calc
      ∑ f : ((i : ι) → α i), ∏ i : ι, μ i (f i)
          = ∑ f ∈ Fintype.piFinset
              (fun i : ι => (Finset.univ : Finset (α i))),
              ∏ i : ι, μ i (f i) := by
            simp
      _ = ∏ i : ι, ∑ a ∈ (Finset.univ : Finset (α i)), μ i a := by
            symm
            exact Finset.prod_univ_sum
              (t := fun i : ι => (Finset.univ : Finset (α i)))
              (f := fun i a => μ i a)
      _ = 1 := by simp [hcoord])
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

**`EconCSLib.pmfProb`**

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
{α : Type u_1} →
  [inst : Fintype α] →
    [inst_1 : DecidableEq α] →
      (μ : PMF α) → (p : α → Prop) → [inst_2 : DecidablePred p] → EconCSLib.pmfExp μ fun a => if p a then 1 else 0
```

### Exact Lean library declaration

```
noncomputable def pmfProb {α : Type*} [Fintype α] [DecidableEq α]
    (μ : PMF α) (p : α → Prop) [DecidablePred p] : ℝ :=
  pmfExp μ (fun a => if p a then 1 else 0)
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

**EconCSLib.pmfProduct**

**Source connection:** no explicit byte-pinned paper source connection is registered.

### Lean-expanded library semantic target

```
(ι : Type u_1) →
  (α : Type u_2) →
    [inst : Fintype ι] →
      [inst_1 : DecidableEq ι] → [inst_2 : Fintype α] → (μ : PMF α) → PMF.ofFintype (fun f => ∏ i, μ (f i)) ⋯
```

### Exact Lean library declaration

```
noncomputable def pmfProduct (ι α : Type*) [Fintype ι] [DecidableEq ι] [Fintype α]
    (μ : PMF α) : PMF (ι → α) :=
  PMF.ofFintype (fun f : ι → α => ∏ i : ι, μ (f i)) (by
    classical
    have hcoord : ∑ a : α, μ a = 1 := by
      rw [← PMF.tsum_coe μ, tsum_fintype]
    calc
      ∑ f : ι → α, ∏ i : ι, μ (f i)
          = ∏ i : ι, ∑ a : α, μ a := by
            symm
            simpa using
              (Finset.prod_univ_sum
                (t := fun _i : ι => (Finset.univ : Finset α))
                (f := fun _i a => μ a))
      _ = 1 := by simp [hcoord])
```

### Recorded source-to-library screening **Verdict:** `not recorded`

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Source claims

## Review row 1

**Source locator:** cited publication, lines 349-350

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0, a fixed positive k, and a shared iid finite
↪  discrete nonnegative conditional-value law having a positive top value, a strictly smaller nonnegative second bound, and
↪  positive law mass both at the top and below it, the source exact min(k,a) top-k objective is the stated two-stage PMF
↪  experiment and every fixed-total optimal allocation sequence has limiting uniform type shares.
```

**Archival source anchor:** cited publication, lines 349-350

### Recorded basis

Approved finite-discrete corrected target and visible nonnegative exact-top-k model convention in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] {Omega : Type u_1} [inst : Fintype Omega] [inst_1 : DecidableEq Omega]
  (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (itemLaw : PMF Omega) (value : Omega → ℝ)
  {xTop xSecond : ℝ} (k : ℕ)
  (seq :
    PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
      (PRPKG24AccuracyDiversity.TopKValueOracle.common T
          (PRPKG24AccuracyDiversity.finiteDiscreteIidTopKExpected Omega itemLaw k value)).toConsumptionModel
        (fun t => (preferenceLaw t).toReal) k),
  0 < k →
    0 < xTop →
      0 ≤ xSecond →
        xSecond ≤ xTop →
          xSecond < xTop →
            (∀ (omega : Omega), 0 ≤ value omega) →
              (∀ (omega : Omega), value omega ≤ xTop) →
                (∀ (omega : Omega), value omega = xTop ∨ value omega ≤ xSecond) →
                  (0 < EconCSLib.pmfProb itemLaw fun omega => value omega = xTop) →
                    (0 < EconCSLib.pmfProb itemLaw fun omega => ¬value omega = xTop) →
                      (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
                        (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
                            ((PRPKG24AccuracyDiversity.TopKValueOracle.common T
                                      (PRPKG24AccuracyDiversity.finiteDiscreteIidTopKExpected Omega itemLaw k
                                        value)).toConsumptionModel
                                  (fun t => (preferenceLaw t).toReal) k).objective
                                a =
                              EconCSLib.pmfExp preferenceLaw fun t =>
                                EconCSLib.pmfExp (EconCSLib.pmfProduct (Fin (a.count t)) Omega itemLaw) fun sample =>
                                  EconCSLib.Probability.sampleTopKSum (fun i => value (sample i)) k) ∧
                          ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
                            Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop (nhds (1 / ↑T))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem1_i_formulaSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 2

**Source locator:** cited publication, lines 351-365

### Verbatim source input

```
[No record available.]
```

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] {beta c M L : ℝ} {k : ℕ} (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T)
  (baseMeasure : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure baseMeasure],
  MeasureTheory.Integrable (fun x => x) baseMeasure →
    (∀ᵐ (y : ℝ) ∂baseMeasure, L ≤ y ∧ y ≤ M) →
      (∀ᵐ (y : ℝ) ∂baseMeasure, 0 ≤ y) →
        0 < M →
          ∀ (f : ℝ → ℝ),
            (baseMeasure = MeasureTheory.volume.withDensity fun y => ENNReal.ofReal (f y)) →
              (∀ (y : ℝ), 0 ≤ f y) →
                Measurable f →
                  0 < beta →
                    0 < c →
                      Filter.Tendsto (fun x => f x / (M - x) ^ (beta - 1)) (nhdsWithin M (Set.Iio M)) (nhds c) →
                        (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
                          0 < k →
                            0 < M - L →
                              ∀
                                (seq :
                                  PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
                                    PRPKG24AccuracyDiversity.boundedIidOrderStatisticConsumptionModel
                                      (fun t => (preferenceLaw t).toReal) k baseMeasure),
                                (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
                                    (PRPKG24AccuracyDiversity.boundedIidOrderStatisticConsumptionModel
                                            (fun t => (preferenceLaw t).toReal) k baseMeasure).objective
                                        a =
                                      EconCSLib.pmfExp preferenceLaw fun t =>
                                        EconCSLib.Probability.expectedSampleTopKSum
                                          (PRPKG24AccuracyDiversity.definition3IidSampleMeasure baseMeasure (a.count t))
                                          k) ∧
                                  ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
                                    Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
                                      (nhds
                                        ((preferenceLaw t).toReal ^ (beta / (beta + 1)) /
                                          ∑ i, (preferenceLaw i).toReal ^ (beta / (beta + 1))))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem1_ii_formulaSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded
**Reviewer check:** ☐ Matches source input
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 3

**Source locator:** cited publication, lines 431-433

### Verbatim source input

```
[No record available.]
```

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (lambda : ℝ) (k : ℕ)
  (hlambda_pos : 0 < lambda),
  0 < k →
    (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
      ∀
        (seq :
          PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
            (PRPKG24AccuracyDiversity.exponentialTopKOrderStatisticOracle T lambda k).toConsumptionModel
              (fun t => (preferenceLaw t).toReal) k),
        (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
            ((PRPKG24AccuracyDiversity.exponentialTopKOrderStatisticOracle T lambda k).toConsumptionModel
                    (fun t => (preferenceLaw t).toReal) k).objective
                a =
              EconCSLib.pmfExp preferenceLaw fun t =>
                EconCSLib.Probability.expectedSampleTopKSum
                  ((PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure
                    (a.count t))
                  k) ∧
          ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
            Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
              (nhds ((preferenceLaw t).toReal ^ 1 / ∑ i, (preferenceLaw i).toReal ^ 1))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem1_iii_formulaSpec_proof
```

### Recorded source-to-Spec screening  Verdict: `not recorded`

**Reason:** not recorded
**Reviewer check:** ☐ Matches source input
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 4

**Source locator:** cited publication, lines 434-437

### Verbatim source input

```
[No record available.]
```

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) {k : ℕ} {alpha : ℝ},
  1 < alpha →
    0 < k →
      (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
        ∀
          (seq :
            PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
              PRPKG24AccuracyDiversity.paretoIidOrderStatisticConsumptionModel (fun t => (preferenceLaw t).toReal) k
                alpha),
          (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
              (PRPKG24AccuracyDiversity.paretoIidOrderStatisticConsumptionModel (fun t => (preferenceLaw t).toReal) k
                      alpha).objective
                  a =
                EconCSLib.pmfExp preferenceLaw fun t =>
                  EconCSLib.Probability.expectedSampleTopKSum
                    (PRPKG24AccuracyDiversity.paretoIidSampleMeasure alpha (a.count t)) k) ∧
            ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
              Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
                (nhds
                  ((preferenceLaw t).toReal ^ (alpha / (alpha - 1)) /
                    ∑ i, (preferenceLaw i).toReal ^ (alpha / (alpha - 1))))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem1_iv_formulaSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 5

**Source locator:** cited publication, lines 438-441

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a finite type space with a preferred-type PMF, one common iid conditional-value probability law D whose identity is
↪  integrable and whose mean is nonnegative, and a selected type maximizing the PMF atom mass, the source all-consumed objective
↪  is the stated selected-type and finite-iid-sample expectation and the all-on allocation at that type is an optimal selected
↪  allocation at every total.
```

**Archival source anchor:** cited publication, lines 437-438

### Recorded basis

Approved selected-optimizer/tie convention and recorded nonnegative-mean Theorem 1(v) correction in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (D : MeasureTheory.Measure ℝ)
  [MeasureTheory.IsProbabilityMeasure D],
  MeasureTheory.Integrable (fun x => x) D →
    0 ≤ ∫ (x : ℝ), x ∂D →
      ∀ (N : ℕ) (best : PRPKG24AccuracyDiversity.ItemType T),
        (∀ (t : PRPKG24AccuracyDiversity.ItemType T), (preferenceLaw t).toReal ≤ (preferenceLaw best).toReal) →
          (PRPKG24AccuracyDiversity.iidAllConsumedSourceModel preferenceLaw D).IsOptimalAtTotal N
            (PRPKG24AccuracyDiversity.allOnTypeAllocation N best)
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem1_v_selected_maximizer_conclusionSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 6

**Source locator:** cited publication, lines 443-457

### Verbatim source input

```
[No record available.]
```

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] {k : ℕ} (likelihood : PRPKG24AccuracyDiversity.ItemType T → ℝ) (gamma : ℝ),
  0 < k →
    0 ≤ gamma →
      (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < likelihood t) →
        ∃ M,
          PRPKG24AccuracyDiversity.Corollary1SourceIidFamily likelihood gamma k M ∧
            ∀ (seq : PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x => M)
              (t : PRPKG24AccuracyDiversity.ItemType T),
              Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
                (nhds (likelihood t ^ gamma / ∑ i, likelihood i ^ gamma))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.corollary1Spec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 7

**Source locator:** cited publication, lines 459-516

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For the iid Uniform([0,1]) source model with positive type likelihoods and every displayed-admissible positive top-k sequence,
↪  every fixed-total optimal allocation has representation error at most (2*m+1)/n from the square-root likelihood profile, and
↪  every optimal allocation sequence converges to that profile.
```

**Archival source anchor:** cited publication, lines 459-485

### Recorded basis

Approved corrected Proposition 2 finite constant and unchanged asymptotic conclusion in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [inst : NeZero T] (likelihood : PRPKG24AccuracyDiversity.ItemType T → ℝ) (kseq : ℕ → ℕ),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < likelihood t) →
    (∀ (N : ℕ), 0 < N → 0 < kseq N) →
      (∀ (N : ℕ), 0 < N → ↑(kseq N) + 1 ≤ ↑N * PRPKG24AccuracyDiversity.uniformSqrtMinShare likelihood - ↑T) →
        ∀
          (seq :
            PRPKG24AccuracyDiversity.OptimalAllocationSequence fun N =>
              PRPKG24AccuracyDiversity.uniformTopKConsumptionModel likelihood (kseq N)),
          (∀ (N : ℕ),
              0 < N →
                ∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
                  (PRPKG24AccuracyDiversity.uniformTopKConsumptionModel likelihood (kseq N)).IsOptimalAtTotal N a →
                    (PRPKG24AccuracyDiversity.PaperInterface.proposition2SqrtProfile likelihood).Approx a
                      ((2 * ↑(Fintype.card (PRPKG24AccuracyDiversity.ItemType T)) + 1) / ↑N)) ∧
            seq.toAllocationSequence.ConvergesToProfile
              (PRPKG24AccuracyDiversity.PaperInterface.proposition2SqrtProfile likelihood)
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.proposition2_corrected_finite_and_sequence_sourceSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 8

**Source locator:** cited publication, lines 518-535

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0 and, conditional on each selected type, iid
↪  Bernoulli(q) values for one common q in (0,1), the source top-one objective is the stated selected-type and
↪  finite-iid-Bernoulli two-stage expectation and every fixed-total optimal allocation sequence has limiting uniform type shares.
```

**Archival source anchor:** cited publication, lines 518-521

### Recorded basis

Approved corrected Corollary 3 q-in-(0,1) target and visible positive-PMF-support convention in docs/BERNOULLI_MAIN_RESULTS_CORRECTED_TARGET_APPROVAL_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (B : PRPKG24AccuracyDiversity.BernoulliSatisfactionModel T)
  (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), B.likelihood t = (preferenceLaw t).toReal) →
    (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < B.successProb t) →
      (∀ (t : PRPKG24AccuracyDiversity.ItemType T), B.successProb t < 1) →
        (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
          (∀ (i j : PRPKG24AccuracyDiversity.ItemType T), B.successProb i = B.successProb j) →
            PRPKG24AccuracyDiversity.ConsumptionModel.AsymptoticHomogeneity (fun x => B.toConsumptionModel)
              (PRPKG24AccuracyDiversity.uniformProfile T)
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.corollary3_iid_bernoulli_conclusionSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 9

**Source locator:** cited publication, lines 540-543

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0 and, conditional on the selected type,
↪  independent rank-varying Bernoulli values satisfying P[X_i^(t)=1] = c*((i+1)+d)^(-alpha), 0 <= alpha < 1, c > 0, d >= 0, and
↪  c*(1+d)^(-alpha) < 1, the source top-one objective is the stated two-stage PMF experiment and every optimal top-one
↪  allocation sequence has limiting uniform type shares.
```

**Archival source anchor:** cited publication, lines 537-541

### Recorded basis

Approved Theorem 2 independent rank-varying Bernoulli model correction in docs/THEOREM2_MODEL_CORRECTION_
2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (alpha c d : ℝ)
  (halpha_nonneg : 0 ≤ alpha),
  alpha < 1 →
    ∀ (hc_pos : 0 < c) (hd_nonneg : 0 ≤ d)
      (hfirst_lt_one : PRPKG24AccuracyDiversity.decayingBernoulliSuccess c d alpha 0 < 1),
      (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
        ∀
          (seq :
            PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
              PRPKG24AccuracyDiversity.decayingBernoulliTopOneConsumptionModel (fun t => (preferenceLaw t).toReal) c d
                alpha),
          (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
              (PRPKG24AccuracyDiversity.decayingBernoulliTopOneConsumptionModel (fun t => (preferenceLaw t).toReal) c d
                      alpha).objective
                  a =
                EconCSLib.pmfExp preferenceLaw fun t =>
                  EconCSLib.pmfExp
                    (PRPKG24AccuracyDiversity.decayingBernoulliFiniteLaw c d alpha (a.count t) ⋯ hd_nonneg halpha_nonneg
                      ⋯)
                    PRPKG24AccuracyDiversity.rankBernoulliFiniteTopOneSampleValue) ∧
            ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
              Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop (nhds (1 / ↑T))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem2_i_corrected_pmf_source_model_endpointSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 10

**Source locator:** cited publication, lines 544-546

## Verbatim source input

```
[No record available.]
```

## Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0 and, conditional on the selected type,
↪  independent rank-varying Bernoulli values satisfying P[X_i^(t)=1] = c*((i+1)+d)^(-1), c > 0, d >= 0, and c*(1+d)^(-1) < 1,
↪  the source top-one objective is the stated two-stage PMF experiment and every optimal top-one allocation sequence has shares
↪  converging to p_t^(1/(1+c)) / sum_i p_i^(1/(1+c)).
```

**Archival source anchor:** cited publication, lines 537-545

## Recorded basis

Approved Theorem 2 independent rank-varying Bernoulli model correction in docs/THEOREM2_MODEL_CORRECTION_2026-07-27.md.

## Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (c d : ℝ) (hc_pos : 0 < c)
  (hd_nonneg : 0 ≤ d) (hfirst_lt_one : PRPKG24AccuracyDiversity.decayingBernoulliSuccess c d 1 0 < 1),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
    ∀
      (seq :
        PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
          PRPKG24AccuracyDiversity.decayingBernoulliTopOneConsumptionModel (fun t => (preferenceLaw t).toReal) c d 1),
      (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
          (PRPKG24AccuracyDiversity.decayingBernoulliTopOneConsumptionModel (fun t => (preferenceLaw t).toReal) c d
                1).objective
              a =
            EconCSLib.pmfExp preferenceLaw fun t =>
              EconCSLib.pmfExp
                (PRPKG24AccuracyDiversity.decayingBernoulliFiniteLaw c d 1 (a.count t) ⋯ hd_nonneg
                  PRPKG24AccuracyDiversity.PaperInterface.theorem2_ii_corrected_pmf_source_model_endpointSpec._proof_1
                  ⋯)
                PRPKG24AccuracyDiversity.rankBernoulliFiniteTopOneSampleValue) ∧
        ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
          Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
            (nhds ((preferenceLaw t).toReal ^ (1 / (1 + c)) / ∑ i, (preferenceLaw i).toReal ^ (1 / (1 + c))))
```

## Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem2_ii_corrected_pmf_source_model_endpointSpec_proof
```

## Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 11

**Source locator:** cited publication, lines 547-551

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0 and, conditional on the selected type,
↪  independent rank-varying Bernoulli values satisfying P[X_i^(t)=1] = c*((i+1)+d)^(-alpha), alpha > 1, c > 0, d >= 0, and
↪  c*(1+d)^(-alpha) < 1, the source top-one objective is the stated two-stage PMF experiment and every optimal top-one
↪  allocation sequence has shares converging to p_t^(1/alpha) / sum_i p_i^(1/alpha).
```

**Archival source anchor:** cited publication, lines 537-548

### Recorded basis

Approved Theorem 2 independent rank-varying Bernoulli model correction in docs/THEOREM2_MODEL_CORRECTION_
2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (alpha c d : ℝ)
  (halpha_gt_one : 1 < alpha) (hc_pos : 0 < c) (hd_nonneg : 0 ≤ d)
  (hfirst_lt_one : PRPKG24AccuracyDiversity.decayingBernoulliSuccess c d alpha 0 < 1),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
    ∀
      (seq :
        PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
          PRPKG24AccuracyDiversity.decayingBernoulliTopOneConsumptionModel (fun t => (preferenceLaw t).toReal) c d
            alpha),
      (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
          (PRPKG24AccuracyDiversity.decayingBernoulliTopOneConsumptionModel (fun t => (preferenceLaw t).toReal) c d
                  alpha).objective
              a =
            EconCSLib.pmfExp preferenceLaw fun t =>
              EconCSLib.pmfExp
                (PRPKG24AccuracyDiversity.decayingBernoulliFiniteLaw c d alpha (a.count t) ⋯ hd_nonneg ⋯ ⋯)
                PRPKG24AccuracyDiversity.rankBernoulliFiniteTopOneSampleValue) ∧
        ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
          Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
            (nhds ((preferenceLaw t).toReal ^ (1 / alpha) / ∑ i, (preferenceLaw i).toReal ^ (1 / alpha)))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem2_iii_corrected_pmf_source_model_endpointSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 12

**Source locator:** cited publication, lines 552-564

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0 and, conditional on the selected type,
↪  independent rank-varying Bernoulli values satisfying P[X_i^(t)=1] = c*((i+1)+d)^(-alpha), alpha >= 0, c >= 0, d >= 0, and
↪  c*(1+d)^(-alpha) <= 1: the source all-consumed objective is the stated two-stage PMF experiment; if alpha > 0 and c > 0,
↪  every optimal all-consumed allocation sequence has shares converging to p_t^(1/alpha) / sum_i p_i^(1/alpha); if alpha = 0,
↪  the corrected selected-optimizer convention yields an all-on allocation at a likelihood-maximizing type.
```

**Archival source anchor:** cited publication, lines 537-553

### Recorded basis

Approved Theorem 2 independent rank-varying Bernoulli model correction in docs/THEOREM2_MODEL_CORRECTION_
2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T) (alpha c d : ℝ)
  (halpha_nonneg : 0 ≤ alpha) (hc_nonneg : 0 ≤ c) (hd_nonneg : 0 ≤ d)
  (hfirst_le_one : PRPKG24AccuracyDiversity.decayingBernoulliSuccess c d alpha 0 ≤ 1),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
    (0 < alpha →
        ∀ (hc_pos : 0 < c)
          (seq :
            PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x =>
              PRPKG24AccuracyDiversity.decayingBernoulliAllConsumedConsumptionModel (fun t => (preferenceLaw t).toReal)
                c d alpha),
          (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
              (PRPKG24AccuracyDiversity.decayingBernoulliAllConsumedConsumptionModel (fun t => (preferenceLaw t).toReal)
                      c d alpha).objective
                  a =
                EconCSLib.pmfExp preferenceLaw fun t =>
                  EconCSLib.pmfExp
                    (PRPKG24AccuracyDiversity.decayingBernoulliFiniteLaw c d alpha (a.count t) ⋯ hd_nonneg halpha_nonneg
                      hfirst_le_one)
                    PRPKG24AccuracyDiversity.rankBernoulliFiniteAllConsumedSampleValue) ∧
            ∀ (t : PRPKG24AccuracyDiversity.ItemType T),
              Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
                (nhds ((preferenceLaw t).toReal ^ (1 / alpha) / ∑ i, (preferenceLaw i).toReal ^ (1 / alpha)))) ∧
      (alpha = 0 →
        ∀ (N : ℕ) (best : PRPKG24AccuracyDiversity.ItemType T),
          (∀ (t : PRPKG24AccuracyDiversity.ItemType T), (preferenceLaw t).toReal ≤ (preferenceLaw best).toReal) →
            (∀ (a : PRPKG24AccuracyDiversity.CountAllocation T),
                (PRPKG24AccuracyDiversity.decayingBernoulliAllConsumedConsumptionModel
                        (fun t => (preferenceLaw t).toReal) c d alpha).objective
                    a =
                  EconCSLib.pmfExp preferenceLaw fun t =>
                    EconCSLib.pmfExp
                      (PRPKG24AccuracyDiversity.decayingBernoulliFiniteLaw c d alpha (a.count t) hc_nonneg hd_nonneg
                        halpha_nonneg hfirst_le_one)
                      PRPKG24AccuracyDiversity.rankBernoulliFiniteAllConsumedSampleValue) ∧
              (PRPKG24AccuracyDiversity.decayingBernoulliAllConsumedConsumptionModel (fun t => (preferenceLaw t).toReal)
                    c d alpha).IsOptimalAtTotal
                N (PRPKG24AccuracyDiversity.allOnTypeAllocation N best))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem2_iv_corrected_pmf_source_model_endpointSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 13

**Source locator:** cited publication, lines 605-662

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with a preferred-type PMF having every atom p_t > 0 and, conditional on each selected type t,
↪  iid Bernoulli values with q_t in (0,1), the source top-one objective is the stated selected-type and finite-iid-Bernoulli
↪  two-stage expectation and every fixed-total optimal allocation sequence has limiting type share (1 / log(1 / (1 - q_t))) /
↪  sum_i (1 / log(1 / (1 - q_i))) for every t.
```

**Archival source anchor:** cited publication, lines 605-619

### Recorded basis

Approved corrected Theorem 3 q_t-in-(0,1) target and visible positive-PMF-support convention in docs/BERNOULLI_MAIN_RESULTS_CORRECTED_TARGET_APPROVAL_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {T : ℕ} [NeZero T] (B : PRPKG24AccuracyDiversity.BernoulliSatisfactionModel T)
  (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), B.likelihood t = (preferenceLaw t).toReal) →
    (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < B.successProb t) →
      (∀ (t : PRPKG24AccuracyDiversity.ItemType T), B.successProb t < 1) →
        (∀ (t : PRPKG24AccuracyDiversity.ItemType T), 0 < (preferenceLaw t).toReal) →
          ∀ (seq : PRPKG24AccuracyDiversity.OptimalAllocationSequence fun x => B.toConsumptionModel)
            (t : PRPKG24AccuracyDiversity.ItemType T),
            Filter.Tendsto (fun N => (seq.allocation N).representation t) Filter.atTop
              (nhds
                (PRPKG24AccuracyDiversity.theorem3LogShareWeight B t /
                  ∑ i, PRPKG24AccuracyDiversity.theorem3LogShareWeight B i))
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem3_log_share_conclusionSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 14

**Source locator:** cited publication, lines 619

### Verbatim source input

```
[No record available.]
```

### Expanded PaperInterface specification

```
∀ {T : ℕ} (B : PRPKG24AccuracyDiversity.BernoulliSatisfactionModel T)
  (preferenceLaw : PRPKG24AccuracyDiversity.SourcePreferenceLaw T),
  (∀ (t : PRPKG24AccuracyDiversity.ItemType T), B.likelihood t = (preferenceLaw t).toReal) →
    PRPKG24AccuracyDiversity.assumption_bernoulli_success_probability_range B →
      ∀ (N : ℕ) (best : PRPKG24AccuracyDiversity.ItemType T),
        (∀ (t : PRPKG24AccuracyDiversity.ItemType T),
            B.likelihood t * B.successProb t ≤ B.likelihood best * B.successProb best) →
          (PRPKG24AccuracyDiversity.bernoulliAllConsumedModel B).IsOptimalAtTotal N
            (PRPKG24AccuracyDiversity.allOnTypeAllocation N best)
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.theorem3_all_consumed_argmax_conclusionSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded
**Reviewer check:** ☐ Matches source input
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 15

**Source locator:** cited publication, lines 703-729

### Verbatim source input

```
Lemma 1. If D has support bounded from above by M with pdf fD such that limx→M (Mf−x)
β−1 = c for some

β, c > 0, then lima→∞ M k−h(a)
= 1.
−1
Ba

β

Combining Lemma 1 with Lemma D.1(ii), with σ = − β1 , we show that for D as in Theorem 1(ii),
β

(n)

a
p β+1
lim t = P t
β .
n→∞ n
m
β+1
i=1 pi

5

(11)
```

### Expanded PaperInterface specification

```
∀ {beta c M L : ℝ} {k : ℕ} (baseMeasure : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure baseMeasure],
  (∀ᵐ (y : ℝ) ∂baseMeasure, L ≤ y ∧ y ≤ M) →
    0 < M - L →
      ∀ (f : ℝ → ℝ),
        (baseMeasure = MeasureTheory.volume.withDensity fun y => ENNReal.ofReal (f y)) →
          (∀ (y : ℝ), 0 ≤ f y) →
            Measurable f →
              MeasureTheory.Integrable f MeasureTheory.volume →
                0 < beta →
                  0 < c →
                    Filter.Tendsto (fun u => f (M - u) / (c * u ^ (beta - 1))) (nhdsWithin 0 (Set.Ioi 0)) (nhds 1) →
                      0 < k →
                        EconCSLib.Math.AsymptoticEquivalent
                          (fun a =>
                            ↑k * M -
                              PRPKG24AccuracyDiversity.orderStatisticTopKSumFromMean
                                (PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq fun a =>
                                  MeasureTheory.Measure.pi fun x => baseMeasure)
                                k a)
                          fun a =>
                          (∑ q, PRPKG24AccuracyDiversity.boundedLemmaD2LimitCoeff beta c ↑q.snd) *
                            PRPKG24AccuracyDiversity.boundedTailScale beta a
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemma1_bounded_topk_loss_asymptotic_of_pdf_sourceSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 16

**Source locator:** cited publication, lines 1122-1145

### Verbatim source input

```
Proposition 4. Consider a non-constant function p(u, v) : Sd × Sd → (0, 1], interpreted as the probability
that item v does not satisfy a user with preference u, such that p(u, v) = q(||u − v||) can be expressed as
a function of the cosine distance between u and v. Then let μ be a probability measure with support Sd ,
representing the distribution of user preferences. Then for a probability measure α on Sd , define
[U+0014 control character] Z
[U+0015 control character]
Z
1
log
μ(u) exp n
α(v) log p(u, v) dv du.
(17)
Γ(α) = lim
n→∞ n
Sd
Sd
Then
Γ(π) ∈ inf Γ(α),

(18)

α

where π is the uniform probability measure over Sd .
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
On the source unit-sphere model, with a full-support preference probability measure and a continuous nonconstant radial kernel p
↪  with 0 < p <= 1 on realized distances, the uniform probability profile pi satisfies Gamma(pi) <= Gamma(alpha) for every
↪  probability profile alpha.
```

**Archival source anchor:** cited publication, lines 1122-1145

### Recorded basis

Approved corrected Proposition 4 Equation (18) target and explicit continuity boundary in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ {E : Type u_1} [inst : NormedAddCommGroup E] [inst_1 : InnerProductSpace ℝ E] [inst_2 : MeasurableSpace E]
  [inst_3 : BorelSpace E] [OpensMeasurableSpace (PRPKG24AccuracyDiversity.Proposition4Sphere.UnitSphere E)]
  [inst_5 : FiniteDimensional ℝ E] [inst_6 : Nontrivial E]
  (preferenceMeasure : MeasureTheory.Measure (PRPKG24AccuracyDiversity.Proposition4Sphere.UnitSphere E))
  [MeasureTheory.IsProbabilityMeasure preferenceMeasure] (p : ℝ → ℝ),
  preferenceMeasure.IsOpenPosMeasure →
    Continuous p →
      (∃ r ∈ Set.Icc 0 2, ∃ s ∈ Set.Icc 0 2, p r ≠ p s) →
        (∀ r ∈ Set.Icc 0 2, 0 < p r ∧ p r ≤ 1) →
          ∀ (alpha : MeasureTheory.ProbabilityMeasure (PRPKG24AccuracyDiversity.Proposition4Sphere.UnitSphere E)),
            PRPKG24AccuracyDiversity.Proposition4Sphere.logRadialDistanceProfileSupValue p
                PRPKG24AccuracyDiversity.Proposition4Sphere.sphereVolumeUniformProbabilityMeasure ≤
              PRPKG24AccuracyDiversity.Proposition4Sphere.logRadialDistanceProfileSupValue p alpha
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.proposition4Spec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 17

**Source locator:** cited publication, lines 1298-1322

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with every weight p_t > 0, let a^(n) be an exact fixed-total integer maximizer of sum_t p_t
↪  h(a_t). If h is monotone, B < 0, sigma > 0, A - h(a) is eventually positive, and log(A - h(a)) / (B * a^sigma) tends to 1 as
↪  a tends to infinity, then a_t^(n) / n tends to 1 / m for every t.
```

**Archival source anchor:** cited publication, lines 1298-1322

### Recorded basis

User instruction 2026-07-27: adopt explicit conventions that make a corrected result true, without representing the archival statement as proved.

### Expanded PaperInterface specification

```
∀ {m : ℕ} [NeZero m] {A B sigma : ℝ} {h : ℕ → ℝ} (p : PRPKG24AccuracyDiversity.ItemType m → ℝ)
  (seq : EconCSLib.Allocation.OptimalSequence (fun x => p) fun x x_1 => h),
  (∀ (i : PRPKG24AccuracyDiversity.ItemType m), 0 < p i) →
    Monotone h →
      B < 0 →
        0 < sigma →
          (∀ᶠ (n : ℕ) in Filter.atTop, 0 < PRPKG24AccuracyDiversity.AppendixD1GenericI.saturationGap A h n) →
            Filter.Tendsto
                (fun n => Real.log (PRPKG24AccuracyDiversity.AppendixD1GenericI.saturationGap A h n) / (B * ↑n ^ sigma))
                Filter.atTop (nhds 1) →
              seq.toSequence.ConvergesToProfile fun x => 1 / ↑m
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD1_i_corrected_uniform_optimizer_shares_sourceSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 18

**Source locator:** cited publication, lines 1324-1350

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with every weight p_t > 0, let a^(n) be an exact fixed-total integer maximizer of sum_t p_t
↪  h(a_t). If h is monotone, B > 0, sigma < 0, and (A - h(a)) / (B * a^sigma) tends to 1 as a tends to infinity, then a_t^(n) /
↪  n tends to p_t^(1 / (1 - sigma)) / sum_i p_i^(1 / (1 - sigma)) for every t.
```

**Archival source anchor:** cited publication, lines 1324-1350

### Recorded basis

User instruction 2026-07-27: adopt explicit conventions that make a corrected result true, without representing the archival statement as proved.

### Expanded PaperInterface specification

```
∀ {m : ℕ} [NeZero m] {A B sigma : ℝ} {h : ℕ → ℝ} (p : PRPKG24AccuracyDiversity.ItemType m → ℝ)
  (seq : EconCSLib.Allocation.OptimalSequence (fun x => p) fun x x_1 => h),
  (∀ (i : PRPKG24AccuracyDiversity.ItemType m), 0 < p i) →
    Monotone h →
      0 < B →
        sigma < 0 →
          Filter.Tendsto (fun n => PRPKG24AccuracyDiversity.AppendixD1GenericII.saturationGap A h n / (B * ↑n ^ sigma))
              Filter.atTop (nhds 1) →
            seq.toSequence.ConvergesToProfile (PRPKG24AccuracyDiversity.AppendixD1GenericIIFull.targetShare p sigma)
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD1_ii_corrected_powerTail_optimizer_shares_sourceSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 19

**Source locator:** cited publication, lines 1352-1369

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with every weight p_t > 0 and sum_t p_t = 1, let a^(n) be an exact fixed-total integer maximizer
↪  of sum_t p_t h(a_t). If h is monotone and strictly discretely concave, B > 0, and h(a) - B * log(a) - C tends to 0 as a tends
↪  to infinity, then a_t^(n) / n tends to p_t for every t.
```

**Archival source anchor:** cited publication, lines 1352-1369

### Recorded basis

User instruction 2026-07-27: adopt explicit conventions that make a corrected result true, without representing the archival statement as proved.

### Expanded PaperInterface specification

```
∀ {m : ℕ} [NeZero m] {B C : ℝ} {h : ℕ → ℝ} (p : PRPKG24AccuracyDiversity.ItemType m → ℝ)
  (seq : EconCSLib.Allocation.OptimalSequence (fun x => p) fun x x_1 => h),
  (∀ (i : PRPKG24AccuracyDiversity.ItemType m), 0 < p i) →
    ∑ i, p i = 1 →
      Monotone h →
        PRPKG24AccuracyDiversity.AppendixD1GenericIII.StrictDiscreteConcave h →
          0 < B →
            Filter.Tendsto (PRPKG24AccuracyDiversity.AppendixD1GenericIII.logRemainder h B C) Filter.atTop (nhds 0) →
              seq.toSequence.ConvergesToProfile p
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD1_iii_corrected_probability_optimizer_shares_sourceSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 20

**Source locator:** cited publication, lines 1371-1399

### Verbatim source input

```
[No record available.]
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For a nonempty finite type space with every weight p_t > 0, let a^(n) be an exact fixed-total integer maximizer of sum_t p_t
↪  h(a_t). If h is strictly discretely concave, B > 0, 0 < sigma < 1, and h(a) / (B * a^sigma) tends to 1 as a tends to
↪  infinity, then a_t^(n) / n tends to p_t^(1 / (1 - sigma)) / sum_i p_i^(1 / (1 - sigma)) for every t.
```

**Archival source anchor:** cited publication, lines 1371-1399

### Recorded basis

User instruction 2026-07-27: adopt explicit conventions that make a corrected result true, without representing the archival statement as proved.

### Expanded PaperInterface specification

```
∀ {m : ℕ} [inst : NeZero m] {B sigma : ℝ} {h : ℕ → ℝ} (p : Fin m → ℝ)
  (seq : EconCSLib.Allocation.OptimalSequence (fun x => p) fun x x_1 => h),
  (∀ (i : Fin m), 0 < p i) →
    0 < B →
      0 < sigma →
        sigma < 1 →
          PRPKG24AccuracyDiversity.AppendixD1GenericIV.StrictDiscreteConcave h →
            Filter.Tendsto (PRPKG24AccuracyDiversity.AppendixD1GenericIV.powerTailQuotient h B sigma) Filter.atTop
                (nhds 1) →
              seq.toSequence.ConvergesToProfile (PRPKG24AccuracyDiversity.AppendixD1GenericIV.targetShare p sigma)
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD1_iv_corrected_powerTail_optimizer_shares_sourceSpec_proof
```

### Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 21

**Source locator:** cited publication, lines 2264-2287

### Verbatim source input

```
Lemma D.2. For β > 0,

Z ∞[U+0012 control character] [U+0013 control character]
1
a
GX (x)j (1 − GX (x))a−j dx ∝ a− β .
j
0

(97)

fX (x)
gX (x)
= lim −
=1
lim
x→M c(M − x)β−1
x→0+ cxβ−1

(98)

(1 − ϵ)cxβ−1 ≤ gX (x) ≤ (1 + ϵ)cxβ−1

(99)
```

### Expanded PaperInterface specification

```
∀ {beta c M L : ℝ} {j : ℕ} (baseMeasure : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure baseMeasure],
  (∀ᵐ (y : ℝ) ∂baseMeasure, L ≤ y ∧ y ≤ M) →
    0 < M - L →
      ∀ (f : ℝ → ℝ),
        (baseMeasure = MeasureTheory.volume.withDensity fun y => ENNReal.ofReal (f y)) →
          (∀ (y : ℝ), 0 ≤ f y) →
            Measurable f →
              MeasureTheory.Integrable f MeasureTheory.volume →
                0 < beta →
                  0 < c →
                    Filter.Tendsto (fun u => f (M - u) / (c * u ^ (beta - 1))) (nhdsWithin 0 (Set.Ioi 0)) (nhds 1) →
                      EconCSLib.Math.AsymptoticEquivalent
                        (PRPKG24AccuracyDiversity.boundedLemmaD2IntegralTerm
                          (EconCSLib.Probability.reflectedCDFMass baseMeasure M) j)
                        fun a =>
                        PRPKG24AccuracyDiversity.boundedLemmaD2LimitCoeff beta c j *
                          PRPKG24AccuracyDiversity.boundedTailScale beta a
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD2_bounded_fixed_rank_integral_asymptotic_of_pdf_sourceSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 22

**Source locator:** cited publication, lines 2778-2785

### Verbatim source input

```
Lemma D.3. For D an exponential distribution with rate parameter λ, so that fX (x) = λe−λx for λ > 0,
lim μD (a − i, a) − log a − B(j) = 0

a→∞

(127)

for a constant B(j) > 0. Moreover, μD (a − i, a) is strictly concave.
```

### Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For an iid exponential law with rate lambda > 0 and each fixed valid upper rank r, mu(q-r,q) = (H_q-H_r)/lambda for q>r; after
↪  subtracting (log q)/lambda it tends to (EulerMascheroniConstant-H_r)/lambda, and the fixed-rank forward marginal is
↪  eventually strictly diminishing on valid ranks.
```

**Archival source anchor:** cited publication, lines 2778-2785

### Recorded basis

Approved corrected Lemma D.3 scale, constant, and valid-rank target in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

### Expanded PaperInterface specification

```
∀ (lambda : ℝ) (hlambda_pos : 0 < lambda) (r : ℕ),
  (∀ (q : ℕ),
      r < q →
        PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
            (fun a => (PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure a)
            (q - r) q =
          1 / lambda * (PRPKG24AccuracyDiversity.harmonicReal q - PRPKG24AccuracyDiversity.harmonicReal r)) ∧
    Filter.Tendsto
        (fun q =>
          PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
              (fun a => (PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure a)
              (q - r) q -
            1 / lambda * Real.log ↑q)
        Filter.atTop (nhds (1 / lambda * (Real.eulerMascheroniConstant - PRPKG24AccuracyDiversity.harmonicReal r))) ∧
      ∀ᶠ (q : ℕ) in Filter.atTop,
        PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
              (fun a => (PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure a)
              (q + 2 - r) (q + 2) -
            PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
              (fun a => (PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure a)
              (q + 1 - r) (q + 1) <
          PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
              (fun a => (PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure a)
              (q + 1 - r) (q + 1) -
            PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
              (fun a => (PRPKG24AccuracyDiversity.exponentialDistributionModel lambda hlambda_pos).iidProductMeasure a)
              (q - r) q
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD3_corrected_exponential_fixed_rank_sourceSpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 23

**Source locator:** cited publication, lines 2844-2858

## Verbatim source input

```
Lemma D.4. For D a Pareto distribution with pdf fX (x) = x−α−1 for α > 1,
lim

at →∞

μD (a − i, a)
1

aα

=C

(132)

for a constant C > 0. Moreover, μD (a − i, a) is strictly concave.
```

## Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For the normalized iid Pareto law with density alpha*x^(-alpha-1) on its Pareto support, alpha > 1, and each fixed valid upper
↪  rank, the order-statistic mean has the source power asymptotic with its corrected rank-dependent coefficient and its forward
↪  marginal is eventually strictly diminishing on valid ranks.
```

**Archival source anchor:** cited publication, lines 2844-2858

## Recorded basis

Approved corrected Lemma D.4 normalized-Pareto and valid-rank target in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

## Expanded PaperInterface specification

```
∀ {alpha : ℝ},
  1 < alpha →
    ∀ (r : ℕ),
      (EconCSLib.Math.AsymptoticEquivalent
          (fun q =>
            PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
              (PRPKG24AccuracyDiversity.paretoIidSampleMeasure alpha) (q - r) q)
          fun q => PRPKG24AccuracyDiversity.paretoRankValueCoeff alpha r * ↑q ^ (1 / alpha)) ∧
        ∀ᶠ (q : ℕ) in Filter.atTop,
          PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq (PRPKG24AccuracyDiversity.paretoIidSampleMeasure alpha)
                (q + 2 - r) (q + 2) -
              PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
                (PRPKG24AccuracyDiversity.paretoIidSampleMeasure alpha) (q + 1 - r) (q + 1) <
            PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
                (PRPKG24AccuracyDiversity.paretoIidSampleMeasure alpha) (q + 1 - r) (q + 1) -
              PRPKG24AccuracyDiversity.expectedOrderStatisticMeanSeq
                (PRPKG24AccuracyDiversity.paretoIidSampleMeasure alpha) (q - r) q
```

## Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD4_corrected_pareto_fixed_rank_sourceSpec_proof
```

## Recorded source-to-Spec screening

**Verdict:** `not recorded`

**Reason:** not recorded

**Reviewer check:** ☐ Matches approved corrected target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 24

**Source locator:** cited publication, lines 3819-3840

## Verbatim source input

```
Lemma D.5. Let g1 , g2 , · · · , gm : [0, ∞)m → R be strictly convex functions over the non-negative reals. Then
define
m
X
gt (xt ).
(190)
g(x1 , · · · , xm ) :=
t=1

Pm

*
*
t=1 xt = n, if (x1 , · · · , xm ) is the maximum of g over the non-negative

Then, under the constraint that
reals and (a*1 , · · · , a*m ) is the maximum of g over the non-negative integers, then
⌊x*t ⌋ − m < a*t < ⌊x*t ⌋ + m

(191)

for all t.
```

## Approved corrected review target

The archival source above is not asserted equivalent to this replacement.

```
For separable strictly concave summands on the nonnegative reals, given fixed-sum real and integer maximizers, every coordinate
↪  lies in the strict floor window floor(x_t) < a_t + m and a_t < floor(x_t) + m. The proof basis is derivative-free strict
↪  concavity.
```

**Archival source anchor:** cited publication, lines 3819-3840

## Recorded basis

Approved corrected Lemma D.5 concavity and derivative-free target in docs/CORRECTED_TARGET_APPROVAL_ADDENDUM_2026-07-27.md.

## Expanded PaperInterface specification

```
∀ {κ : Type u_1} [inst : Fintype κ] (g : κ → ℝ → ℝ) (N : ℕ) (x : κ → ℝ) (a : κ → ℕ),
  (∀ (i : κ), StrictConcaveOn ℝ (Set.Ici 0) (g i)) →
    (∀ (i : κ), 0 ≤ x i) →
      ∑ i, x i = ↑N →
        (∀ (z : κ → ℝ),
            (∀ (i : κ), 0 ≤ z i) →
              ∑ i, z i = ↑N →
                PRPKG24AccuracyDiversity.GeneralRounding.objective g z ≤
                  PRPKG24AccuracyDiversity.GeneralRounding.objective g x) →
          ∑ i, a i = N →
            (∀ (b : κ → ℕ),
                ∑ i, b i = N →
                  (PRPKG24AccuracyDiversity.GeneralRounding.objective g fun i => ↑(b i)) ≤
                    PRPKG24AccuracyDiversity.GeneralRounding.objective g fun i => ↑(a i)) →
              ∀ (t : κ), ⌊x t⌋₊ < a t + Fintype.card κ ∧ a t < ⌊x t⌋₊ + Fintype.card κ
```

## Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.lemmaD5Spec_proof
```

## Recorded source-to-Spec screening

**Verdict:** `not recorded`
**Reason:** not recorded
**Reviewer check:** ☐ Matches approved corrected target
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**

# Review row 25

**Source locator:** cited publication, lines 2071-2087

### Verbatim source input

```
Proposition 5. For Xi

~ D,

min{k,a}

h(a) =

X
i=1

µD (a − i + 1, a).

(78)

Recall that µD (i, a) is the expected value of the i-th order statistic of a random variables drawn i.i.d.
from D.
```

### Expanded PaperInterface specification

```
∀ (D : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure D],
  MeasureTheory.Integrable (fun x => x) D →
    ∀ (k a : ℕ),
      EconCSLib.Probability.orderStatisticTopKSumFromMean (PRPKG24AccuracyDiversity.definition3IidOrderStatisticMean D)
          k a =
        EconCSLib.Probability.expectedSampleTopKSum (PRPKG24AccuracyDiversity.definition3IidSampleMeasure D a) k
```

### Lean proof endpoint (built separately)

```
PRPKG24AccuracyDiversity.PaperInterface.proposition5_iid_topk_identitySpec_proof
```

### Recorded source-to-Spec screening **Verdict:** `not recorded`

**Reason:** not recorded
**Reviewer check:** ☐ Matches source input
*If left unchecked, explain the discrepancy or uncertainty below.*
**Reviewer annotation**